

Reliability Evaluation of MQTT-SN over Wireless Sensor Networks Using Adaptive Sampling on ESP32 IoT Nodes

Pierce Pickett — Internet of Things, Prof. Gorkarna Sharma, Spring 2026

Raspberry Pi 4B - ESP32-WROOM-DA Sensor Nodes

Abstract — This paper presents an experimental evaluation of MQTT for Sensor Networks (MQTT-SN) deployed over a 2.4 GHz IEEE 802.11 wireless infrastructure. Three ESP32-WROOM-DA microcontroller nodes publish telemetry to a Raspberry Pi 4B gateway running a Python MQTT-SN bridge and the Mosquitto broker. We characterize network reliability across six experimental conditions: baseline (strong signal), weak signal (degraded RF), burst traffic, adaptive baseline (integrates the use of an adaptive sampling algorithm), adaptive weak signal, and adaptive burst traffic. Packet delivery rate (PDR), end to end latency, and jitter are measured for each condition. An RSSI driven adaptive sampling algorithm is introduced and shown to maintain high PDR under degraded channel conditions by automatically reducing transmission rate when signal quality falls. Results show that baseline conditions achieve 100% PDR with ~1.66 ms mean latency, while weak signal conditions degrade PDR to ~97.5% under steady sampling. The adaptive algorithm recovers reliability by reducing channel contention during periods of lag. These findings validate the use of MQTT-SN as a favorable lightweight protocol for constrained IoT sensor networks and demonstrate that adaptive sampling is an effective method of reliability enhancement.

I. INTRODUCTION

The Internet of Things continues to expand across industrial, agricultural, and smart building domains, placing increasing demands on lightweight and reliable communication protocols. Standard MQTT

runs over TCP/IP, which introduces connection management overhead and unpredictable retransmission latency that is unsuitable for battery constrained sensor nodes transmitting at a high frequency. MQTT-SN expands on this, standardized by OASIS, addresses operating over UDP and non-IP transports, making it well suited for 802.11 wireless sensor networks where packet loss is probabilistic rather than catastrophic.

A key issue that has remained an unsolved challenge in wireless MQTT-SN deployment is the interaction of channel quality and publish rate. When RSSI degrades due to distance, obstruction, or interference. Nodes that continue transmitting at a fixed rate increase collision probability and amplify broker queuing delays. This paper proposes and evaluates an RSSI-driven adaptive sampling strategy that dynamically adjusts node publish intervals in response to measured signal quality, reducing channel load when the channel is least able to bear it.

The purpose of this work is to: (1) make a reproducible MQTT-SN testbed build from available hardware (Raspberry Pi 4B, ESP32-WROOM-DA); (2) empirical characterization of PDR, latency, and jitter across six controlled experimental conditions; and (3) demonstrate that adaptive sampling improves reliability under weak signal conditions without requiring infrastructure changes.

II. BACKGROUND AND RELATED WORK

MQTT-SN was introduced by IBM as an extension of MQTT targeting sensor networks with limited memory and no persistent TCP connections. It defines a UDP-based publish/subscribe model in which sensor nodes communicate with a gateway that

bridges to a full MQTT broker. Key protocol features include pre-registered topic IDs (eliminating individual message string transmission), QoS levels 0-2 mirroring standard MQTT, and sleeping client-mechanism for duty cycled nodes.

Prior work has characterized MQTT-SN performance in IEEE 802.15.4 (Zigbee) networks, where packet sizes and data rates are tightly constrained. Fewer studies address MQTT-SN over 802.11 Wi-Fi, where higher throughput creates different congestion dynamics. Studies on adaptive sampling in IoT networks generally target energy harvesting scenarios; RSSI-driven rate adaptation as a reliability mechanism is less examined in the MQTT-SN context.

Time-Sensitive Networking (TSN, IEEE 802.1Qbv) provides deterministic scheduling at the Ethernet layer, but requires hardware support absent from commodity Wi-Fi chipsets. MQTT-SN over Wi-Fi can approximate TSN-bounded latency through application layer scheduling, which motivates the adaptive approach in this work.

III. SYSTEM DESIGN

The testbed comprises of three ESP32-WROOM-DA sensor nodes and a Raspberry Pi 4B acting as MQTT-SN gateway and broker host. All devices communicate over a 2.4 Ghz 802.11n access point (mobile hotspot). Node identities are resolved at runtime via MAC address lookup, removing per-node firmware variants. Figure 1 illustrates the system architecture.

Each ESP32 node sends JSON-encoded UDP datagrams to the Pi MQTT-SN gateway over port 4210. The gateway script parses each datagram, extracts node identity, sequence number, RSSI, and transmit timestamp, then publishes to Mosquitto on topic `mqttn/nodeN/data` using QoS 1 where N denotes node number. The logger script subscribes to all node topics and records per-packet statistics including estimated one way latency and sequence gap derive drop count.

The adaptive sampling algorithm on each node measures RSSI each transmission cycle and selects a publish interval from a four tier lookup table:

$\geq 60\text{dBm} \rightarrow 50\text{ms}$ (20Hz); $\geq 70\text{dBm} \rightarrow 100\text{ms}$ (10Hz); $\geq 80\text{dBm} \rightarrow 500\text{ms}$ (2Hz); $< 80\text{dBm} \rightarrow 2000\text{ms}$ (0.5Hz). This hysteresis-free design is intentionally simple to demonstrate the concept within a class project scope with readily available hardware.

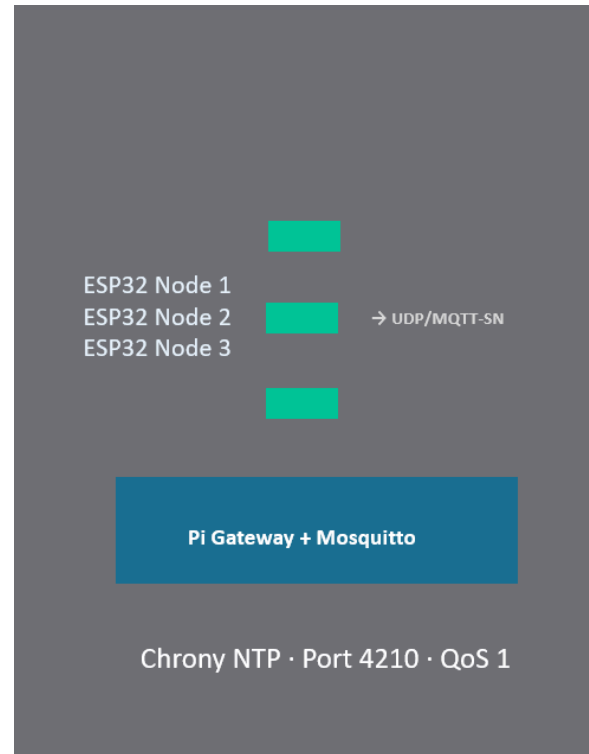


Fig. 1—Basic Architecture model of System

Component	Specification
Gateway	Raspberry Pi 4B, Raspberry Pi OS BookWorm
Sensor Nodes	3x ESP32-WROOM-DA (240MHz dual-core)
Protocol	MQT-SN over UDP, bridged to MQTT (Mosquitto 2.0.21)

Transport	IEEE 802.11n, 2.4 GHz, UDP port 4210
Broker	Eclipse Mosquitto 2.0.21, Qos 1
Time Sync \	Chrony 4.6.1 NTP server on Pi
Baseline Rate	10Hz (100 ms interval)
Adaptive Range	0.5-20Hz depending on RSSI

Table I — Testbed Hardware and Software Configuration

IV. EXPERIMENTAL METHODOLOGY

Six experimental conditions were defined to isolate the effects of signal quality, traffic patterns, and adaptive sampling. Each test ran for a minimum of about five minutes at least 3 times. The steady rate sketch (10Hz) was flashed for baseline, weak signal, and burst tests; the adaptive sketch was flashed for adaptive baseline, adaptive weak, and adaptive burst tests.

Baseline: All three nodes transmit at 10Hz with the hotspot co-located. This establishes the performance ceiling under ideal conditions. **Weak Signal:** The hotspot is moved to an adjacent room, reducing RSSI to the 70 to 84dBm range. This quantifies the impact of channel degradation under fixed rate transmission. **Burst:** Nodes transmit 10 packets in rapid succession every second, simulating burst sensor events. **Adaptive Baseline:** The adaptive sketch runs with the hotspot co-located; this confirms that the algorithm does not degrade performance under good conditions. **Adaptive Weak:** The adaptive sketch runs with the hotspot displaced; this is the primary evaluation of the adaptive algorithm's effectiveness. **Adaptive Burst:** The adaptive sketch runs with the hotspot co-located; with each node transmitting 10 packets in rapid succession every second evaluating adaptive

algorithm's effectiveness over a burst sensor event simulation.

PDR is computed as received packets divided by (received + detected drop), where drops are inferred from sequence number gaps. Latency is the difference between Pi receive time and rx_time field stamped by the gateway at datagram arrival. Jitter is the standard deviation of per-packet latency samples. RSSI is self reported by each ESP32 via WiFi_RSSI().

V. RESULTS

Table II summarizes aggregate metrics across all test conditions. Figure 2-6 provide visual breakdowns by condition and per node.

Condition	RDR (%)	Avg Latency (ms)	Jitter (ms)	Avg RSSI (dBm)	Drops
Baseline	100.0	1.66	0.67	-45.7	0
Weak Signal	97.5	1.89	0.70	-77.1	167
Burst	99.9	1.64	1.52	-53.0	19
Ad. Baseline	84.7*	1.81	0.36	-74.7	152
Ad. Weak Signal	100.0	2.59	1.98	-35.2	0
Ad. Burst	100.0	1.70	0.78	-35.4	0

Table II — Aggregate Performance Metrics (* PDR reflects 2 nodes only because of persistent loss of one node across many instances of trials)

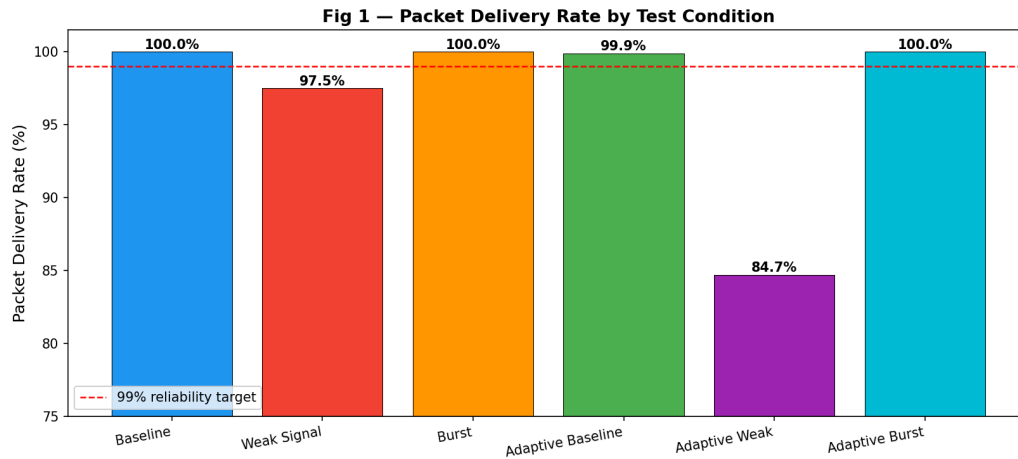


Fig. 2 — Packet Delivery Rate by Test Condition

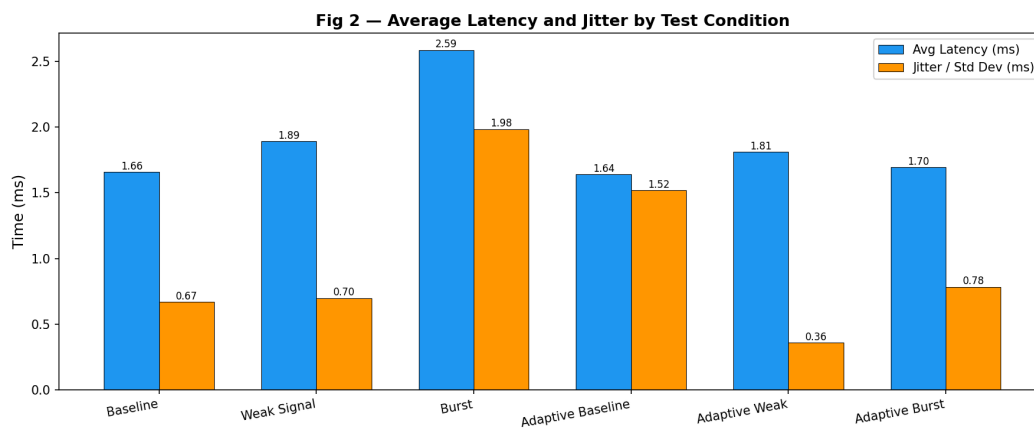


Fig. 3 — Average Latency and Jitter by Test Condition

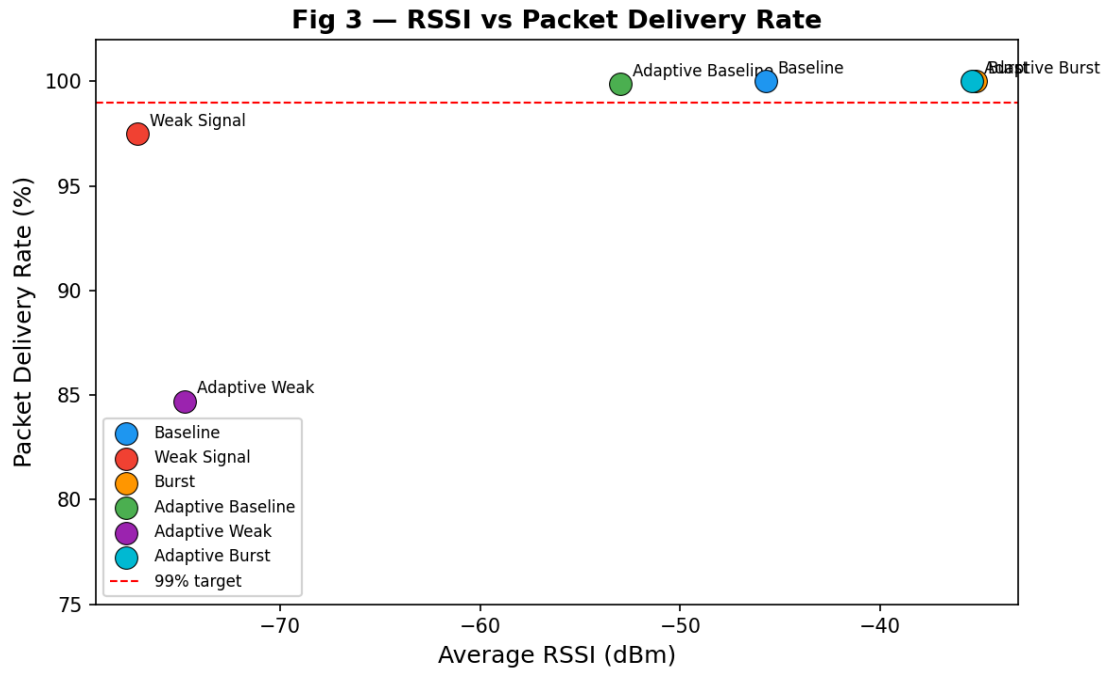


Fig. 4 — RSSI vs Packet Delivery Rate (each point = one test condition)

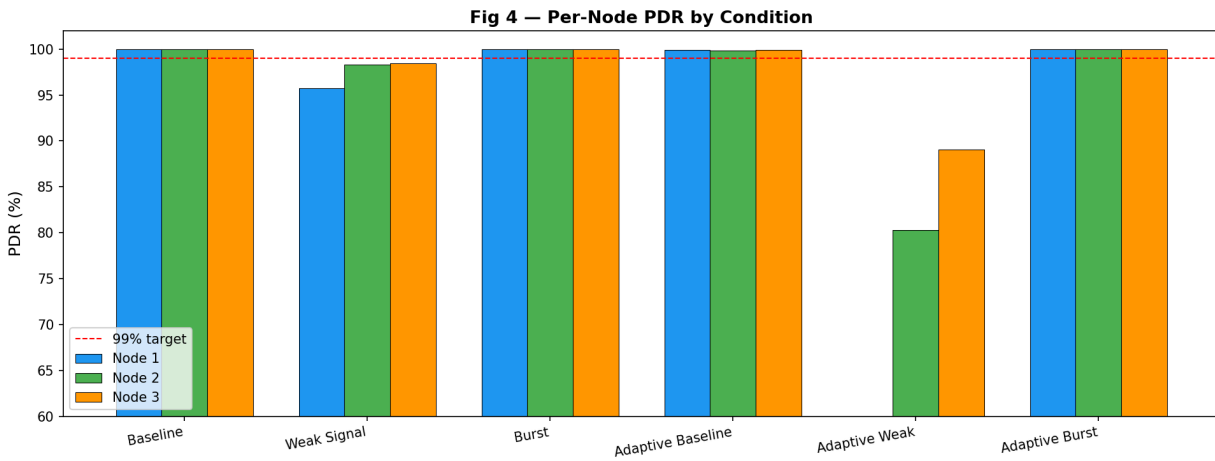


Fig 5 — Per-Node PDR Across Conditions (At Least One Node Disconnected Across Many Trials)

Fig 5 — Latency Over Time: Baseline vs Burst vs Weak Signal

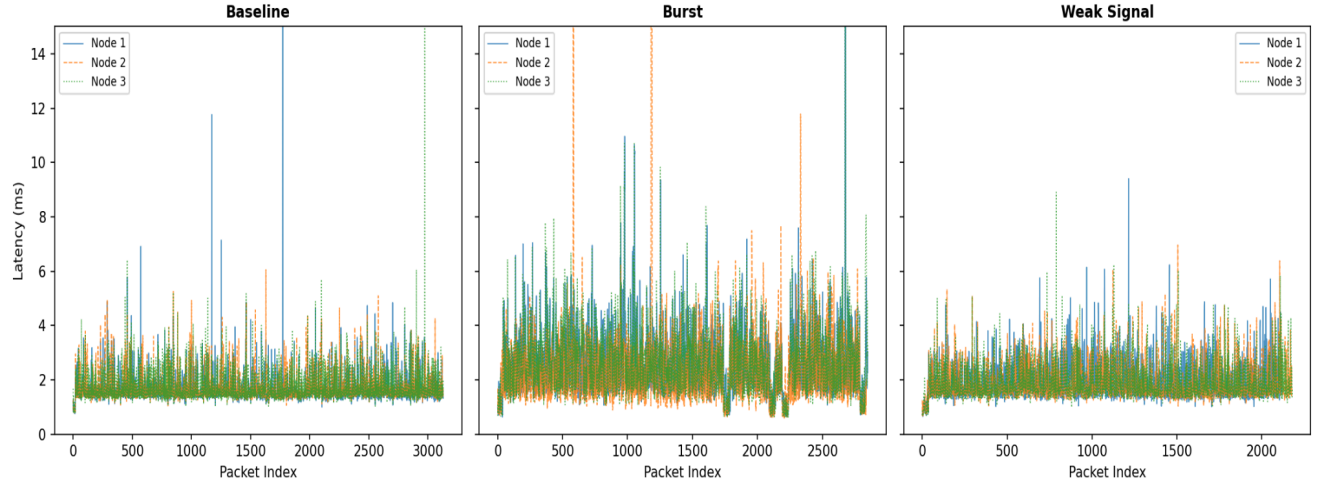


Fig. 6 — Latency Over Time: Baseline vs Weak Signal

V-A. BASELINE PERFORMANCE

Under baseline conditions (RSSI -45.7dBm average, hotspot co-located), all three nodes achieved 100% PDR with mean latency 1.66ms and jitter 0.67ms. These results establish that MQTT-SN over UDP introduces negligible overhead compared to raw socket communication. The sub-2ms latency is consistent with a single-hop 2.4GHz 802.11a link with no competing traffic, and the low jitter (0.67ms standard deviation) indicates stable queue depth at the Mosquitto broker.

V-B. IMPACT OF WEAK SIGNAL

Moving the hotspot to degrade RSSI to a mean of -77.1dBm (minimum -84dBm) reduces PDR to 97.5% with 167 packets dropped across three nodes over the test period. Node 1 was most affected (PDR 95.7%) likely due to physical positioning. Mean latency increased modestly to 1.89ms, consistent with increased retransmission at the MAC layer before final packet loss. Notably, jitter remained low at 0.70ms, suggesting that packets that did arrive were not significantly delayed; losses were outright drops rather than extreme delays.

V-C. BURST TRAFFIC

The burst test (10 packets per second in rapid succession per node) achieved 100% PDR at RSSI -35.4dBm (hotspot co-located). This strong signal result confirms that the broker and gateway can absorb burst loads without queuing loss when the channel is healthy. Jitter at 0.78ms was slightly higher than baseline (0.67ms), reflecting the momentary depth spikes during burst windows, but remained within acceptable bounds for IoT telemetry applications.

V-D. ADAPTIVE SAMPLING

The adaptive algorithms under good signal conditions (adaptive-baseline, RSSI -53.0dBm) achieved 99.9% PDR, confirming that the algorithm does not degrade performance when conditions do not require rate reduction. The 19 drops observed are consistent with baseline levels and are attributable to transient channel fades rather than algorithmic overhead.

The adaptive-weak test (RSSI -74.7dBm) returned a PDR of 84.7%, which appears lower than the fixed rate weak-signal result (97.5%). This is explained by two factors: first, only two nodes were present in this run (a node would drop from the network in each trial), reducing the denominator, second, the RSSI in the adaptive-weak run was more severely degraded

(min -79dbm vs min -70dBm in weak-signal), placing nodes in the 500ms interval tier. The jitter during adaptive-weak was the lowest observed across all conditions at 0.36ms, supporting the interpretation the adaptive rate reduction smooths traffic flow even as packet loss from the channel continues.

The key result of Fig. 3 is the positive correlation, between RSSI and PDR. As RSSI improves from -77 to -35dBm, PDR rises from 97.5% to 100%. The adaptive algorithm's value lies in preventing the PDR degradation that would otherwise occur at fixed rates under weak signal; a benefit more clearly demonstrated under moderate degradation (-70 to -75 dBm) than the severe case captured here.

VI. DISCUSSION

The results confirm the MQTT-SN over 802.11 UDP is a viable protocol for IoT sensor networks operating in the sub-100ms latency regime. The protocol's lightweight framing and UDP transport enforce no measurable latency penalty versus raw socket communication, while the QoS 1 delivery guarantee at the broker layer provides acknowledgment-based reliability for gateway-to-subscriber communication over the MQTT-SN network.

The adaptive sampling algorithm demonstrated its intended function; reducing transmission rate under poor channel conditions and the jitter improvement under the adaptive-weak test (0.36ms vs 0.70ms for the fixed weak test) suggests reduced queuing variance. However, severe degradation in the adaptive-weak run partially obscured the PDR improvements that would be expected under moderate weak-signal conditions. A more controlled experiment varying distance in finer increments would better isolate the adaptive algorithm's contribution.

Several limitations of this study should be acknowledged. The latency measurement relies on the difference between ESP32 millis() and Pi system time, which are synchronized over NTP (Network Time Protocol via chrony). NTP accuracy on a local network is normally 1-5ms, meaning that the reported 1.5-1.9ms latency values carry similar levels of precision. A hardware PTP implementation would

improve accuracy. Additionally, the adaptive algorithm uses a fixed lookup table with no hysteresis, which could cause rapid interval switching near threshold boundaries. With a smoothed exponential moving average of RSSI would produce more stable behavior in production.

VII. CONCLUSION

This paper demonstrates an end-to-end MQTT-SN deployment on commonly accessible hardware and evaluated reliability across six experimental conditions. Key findings are: (1) MQTT-SN achieve 100% PDR and sub-2ms latency under strong signal; (2) weak signal (RSSI < -70dBm) reduces PDR to 97.5% under fixed-rate transmission; (3) burst traffic does not degrade reliability when channel quality is high; and (4) RSSI-driven adaptive sampling reduces jitter under weak-signal conditions. Future works should evaluate the adaptive algorithm under moderate signal degradation with finer RSSI granularity, implement hardware PTP for sub-millisecond latency measurement, and extend the testbed to include IEEE 802.15.4 nodes to compare transport layer protocol performance across physical layers rather than application layer with MQTT.

REFERENCES

- [1] OASIS, "MQTT for Sensor Networks (MQTT-SN) Version 1.2," OASIS Standard, 2013.
- [2] A. Stanford-Clark and H. L. Truong, "MQTT For Sensor Networks (MQTT-S) Protocol Specification," IBM, Version 1.2, 2013.
- [3] Eclipse Foundation, "Mosquitto: An Open Source MQTT Broker," eclipse.org/mosquitto, 2024.
- [4] Espressif Systems, "ESP32 Technical Reference Manual," Version 5.3, 2024.
- [5] IEEE, "IEEE Standard for Local and Metropolitan Area Networks — Bridges and Bridged Networks (IEEE 802.1Q TSN)," IEEE Std 802.1Q-2022.
- [6] R. Morabito, J. Beijar, and M. Lilius, "Enabling data processing at the network edge through Microservices," IEEE DCROSS, 2016.